



گزارش پروژه درس ریزپردازنده
طراحی و پیاده‌سازی سیستم تشخیص گام (Pitch) صوت انسان با روش
خودهمبستگی

مشخصات

نام دانشگاه: دانشگاه شهید بهشتی
نام درس: ریزپردازنده و مدارهای واسط
نام استاد: جناب آقای دکتر عطارزاده
شماره گروه: ۶
اعضای گروه: حامی جهانیان ، سهند مسعودی
تاریخ: ۲۶ بهمن ۱۴۰۴

در این پروژه سیستم نهایی با میکروکنترلر STM32F401RE (برد NUCLEO-F401RE) هدف‌گذاری شده است؛ اما پیاده‌سازی و ارزیابی این گزارش بر مبنای شبیه‌سازی در Proteus انجام شده است.

فهرست مطالب

۳	۱ چکیده
۳	۲ مقدمه و صورت مسئله
۳	۱.۲ تعریف گام و محدوده فرکانسی
۳	۲.۲ هدف پروژه
۳	۳ مدل و مبانی تئوری
۴	۱.۳ تعریف خودهمبستگی سیگنال گسسته
۴	۲.۳ رابطه تاخیر با فرکانس پایه
۴	۳.۳ انتخاب بازه تاخیرها (Lag Range)
۴	۴ معماری سیستم و مشخصات طراحی
۴	۱.۴ نمای کلی مسیر داده
۵	۲.۴ پارامترهای کلیدی انتخاب شده
۵	۵ پیاده سازی نرم افزار
۵	۱.۵ نمونه برداری ADC با Trigger تایمر
۶	۲.۵ پیش پردازش: حذف DC و تشخیص سکوت
۷	۳.۵ محاسبه خودهمبستگی و استخراج lag
۷	۴.۵ روش های مختلف آستانه گذاری (Thresholding) که امتحان شد
۸	۵.۵ نگاشت فرکانس به نت موسیقی
۸	۶ LCD کاراکتری: پیاده سازی بدون کتابخانه و چالش زمان بندی (مهم ترین بخش عملی)
۸	۱.۶ مروری بر LCD 16x2 و پایه ها
۸	۲.۶ دو نسخه LCD: مد ۸ بیتی و مد ۴ بیتی
۹	۳.۶ چرا زمان بندی در Proteus سخت تر شد؟
۹	۴.۶ مرجع زمان بندی و توالی راه اندازی
۱۰	۵.۶ راه حل نهایی زمان بندی: استفاده از DWT برای delay میکروثانیه
۱۱	۷ چالش ها و راه حل ها
۱۱	۱.۷ چالش ۱: زمان بندی LCD (مد ۴ بیتی) — چرا اینقدر مهم بود؟
۱۲	۲.۷ چالش ۲: ADC با Trigger تایمر — «چرا Continuous Conversion باید خاموش باشد؟»
۱۲	۳.۷ چالش ۳: Threshold — سخت ترین بخش DSP پروژه
۱۲	۴.۷ چالش ۴: نبود سخت افزار و تست در Proteus
۱۳	۵.۷ چالش ۵: ساخت و آماده سازی فایل های WAV برای تست
۱۳	۸ تصاویر مفهومی تکمیلی
۱۵	۹ نتایج، ارزیابی و مشاهده خروجی
۱۵	۱.۹ سناریوهای تست پیشنهادی (که با Proteus هم قابل انجام است)

۱ چکیده

هدف این پروژه طراحی یک سیستم تشخیص گام (Pitch) صدای انسان با استفاده از روش خودهمبستگی (Autocorrelation) در حوزه زمان است. سیگنال صوت ورودی با نرخ نمونه‌برداری مشخص دریافت، در بافر ذخیره، پیش‌پردازش (حذف DC و بررسی سکوت) شده و سپس خودهمبستگی آن برای بازه‌ای از تاخیرها محاسبه می‌گردد. با پیدا کردن تاخیر متناظر با قله غالب (یا اولین قله معتبر)، دوره تناوب تقریبی به دست آمده و فرکانس پایه F_0 محاسبه می‌شود. در نهایت با نگاشت F_0 به نزدیک‌ترین نت موسیقی، خروجی روی LCD کاراکتری 16×2 نمایش داده می‌شود.

۲ مقدمه و صورت مسئله

۱.۲ تعریف گام و محدوده فرکانسی

گام (Pitch) در صداهای شبه‌دوره‌ای (به‌خصوص آواهای صدادار انسان) با فرکانس پایه F_0 مرتبط است. در آواهای صدادار، تارهای صوتی مرتعش شده و سیگنال خروجی ماهیت شبه‌تناوبی دارد؛ بنابراین روش‌های حوزه زمان مانند خودهمبستگی معمولاً پاسخ مناسبی می‌دهند. محدوده معمول F_0 برای صدای انسان به طور رایج در بازه تقریبی ۸۵ تا ۳۵۰ هرتز در نظر گرفته می‌شود.

۲.۲ هدف پروژه

هدف، ساخت یک زنجیره کامل از:

- نمونه‌برداری صوت با ADC
- محاسبه خودهمبستگی و استخراج F_0
- نگاشت F_0 به نت موسیقی
- نمایش نت و فرکانس روی LCD 16x2

است. (در این گزارش، به دلیل نبود سخت‌افزار فیزیکی، تمرکز عملی روی شبیه‌سازی Proteus بوده است.)

۳ مدل و مبانی تئوری

ایده اصلی روش خودهمبستگی

خودهمبستگی یعنی سیگنال را با نسخه تاخیردار خودش ضرب داخلی کنیم. برای یک سیگنال شبه‌دوره‌ای، وقتی تاخیر برابر دوره تناوب باشد، هم‌فازی رخ می‌دهد و مقدار خودهمبستگی معمولاً قله می‌زند. پس اگر بتوانیم اولین قله معتبر (یا قله غالب) را پیدا کنیم، می‌توانیم دوره تناوب T_0 و سپس $F_0 = \frac{1}{T_0}$ را تخمین بزنیم.

۱.۳ تعریف خودهمبستگی سیگنال گسسته

برای سیگنال گسسته $x[n]$ با انرژی متناهی، خودهمبستگی بر حسب تاخیر l به صورت زیر تعریف می‌شود:

$$R_x(l) = \sum_{n \in \mathbb{Z}} x[n] x[n-l]$$

در پیاده‌سازی عملی، چون بافر طول محدود دارد، جمع روی بازه محدود انجام می‌شود:

$$R_x(l) = \sum_{n=l}^{N-1} x[n] x[n-l]$$

که N طول بافر است.

۲.۳ رابطه تاخیر با فرکانس پایه

اگر نرخ نمونه‌برداری F_s باشد و تاخیر نمونه‌ای متناظر با دوره تناوب را τ بگیریم:

$$F_0 \approx \frac{F_s}{\tau}$$

در کد پروژه، برای دقت بیشتر خروجی به صورت centi-Hz (یعنی $100 \times \text{Hz}$) محاسبه شده است.

۳.۳ انتخاب بازه تاخیرها (Lag Range)

برای اینکه جستجو فقط روی محدوده فرکانسی صدای انسان انجام شود:

$$\tau_{\min} \approx \left\lfloor \frac{F_s}{F_{\max}} \right\rfloor, \quad \tau_{\max} \approx \left\lceil \frac{F_s}{F_{\min}} \right\rceil$$

در پروژه با $F_s = 8000 \text{ Hz}$ و بازه $\text{Hz}: [85, 350]$

$$\tau_{\min} \approx 23, \quad \tau_{\max} \approx 94$$

که دقیقاً با مقادیر $\text{LAG_MIN}=23$ و $\text{LAG_MAX}=94$ در کد منطبق است.

۴ معماری سیستم و مشخصات طراحی

۱.۴ نمای کلی مسیر داده

مسیر داده در این پروژه به صورت زیر است:

۱. دریافت نمونه‌ها با ADC (با Trigger از تایمر) و ذخیره در بافر،
۲. حذف مؤلفه DC (کم کردن میانگین) و محاسبه انرژی/بیشینه دامنه برای تشخیص سکوت،
۳. محاسبه خودهمبستگی برای lag های محدوده انسان،

۴. استخراج lag متناظر با قله معتبر،
۵. محاسبه F_0 و نگاشت به نزدیک‌ترین نت،
۶. نمایش روی LCD و ارسال گزارش روی UART (برای دیباگ).

۲.۴ پارامترهای کلیدی انتخاب‌شده

جدول ۱: پارامترهای طراحی اصلی

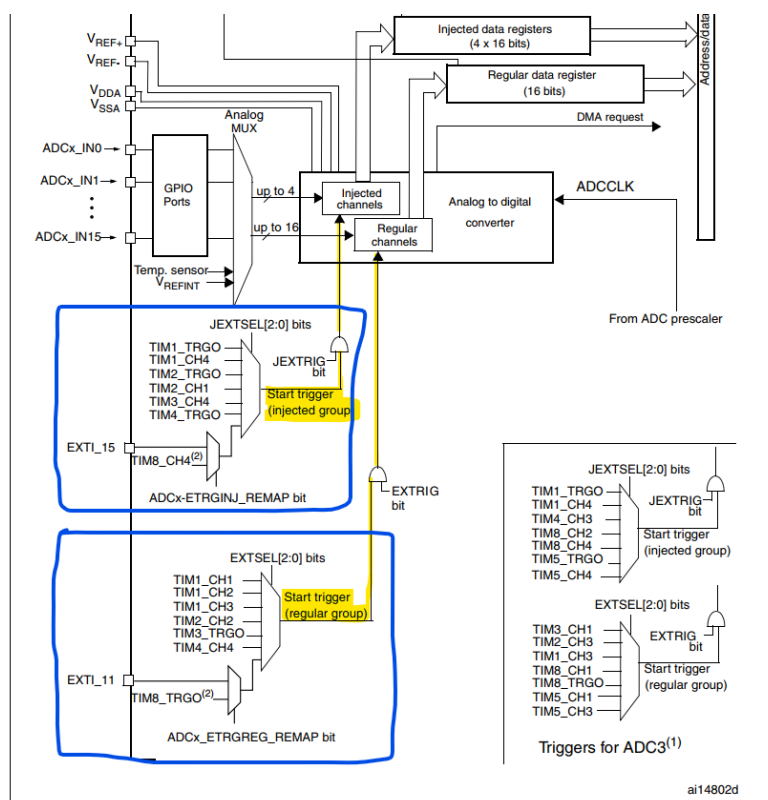
پارامتر	مقدار	علت/توضیح
نرخ نمونه‌برداری F_s	8000 Hz	مصالحه بین دقت و هزینه محاسباتی، مناسب بازه 85–350Hz
طول بافر N	۳۲۰ نمونه	افزایش پایداری خودهمبستگی و کاهش خطا در نویز
بازه lag	۲۳ تا ۹۴	مطابق محدوده فرکانس انسان
رزولوشن ADC	۱۲ بیت	رزولوشن استاندارد و کافی برای تشخیص گام در این سطح
نمایش فرکانس	centi-Hz	نمایش با دو رقم اعشار بدون شناور
آستانه سکوت (MaxAbs)	۲۵	حذف فریم‌های کم‌دامنه/نویز
آستانه سکوت (Energy)	۲۰۰۰۰۰	جلوگیری از تشخیص غلط در سکوت و نویز کم

۵ پیاده‌سازی نرم‌افزار

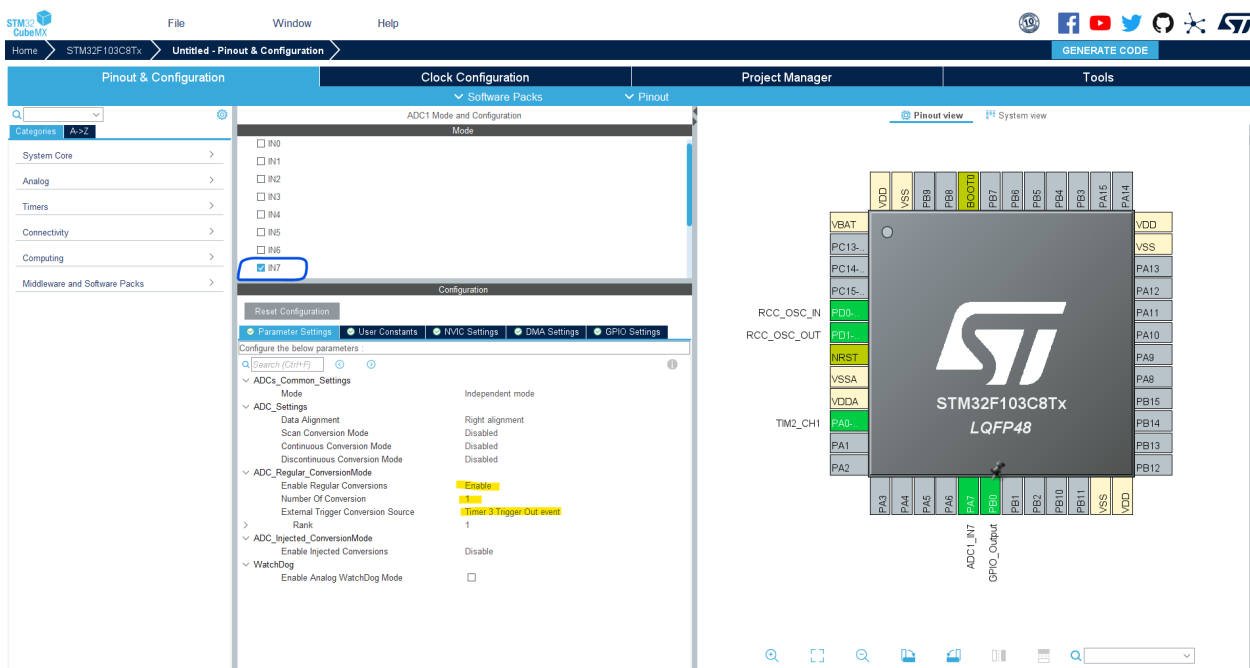
۱.۵ نمونه‌برداری ADC با Trigger تایمر

در این پروژه ADC با رخداد TRGO تایمر تحریک می‌شود. نکته مهم این است که وقتی ADC از بیرون Trigger می‌گیرد، باید:

- **Conversion Continuous** خاموش باشد،
- **Trigger External** و **Edge** درست تنظیم شود،
- **EOC (End of Conversion)** برای هر نمونه فعال باشد.



شکل ۱: منابع Trigger خارجی ADC (نمودار مفهومی)



شکل ۲: نمونه تنظیم Trigger تایمری برای ADC در CubeMX

۲.۵ پیش پردازش: حذف DC و تشخیص سکوت

سیگنال ADC معمولاً دارای آفست DC است (به خصوص اگر ورودی حول نیم ولتاژ مرجع بایاس شده باشد). بنابراین در کد:

- ابتدا میانگین نمونه‌ها محاسبه می‌شود،
 - از هر نمونه کم می‌شود تا سیگنال حول صفر قرار بگیرد،
 - سپس maxabs و energy محاسبه شده و اگر هرکدام پایین باشد، فریم سکوت در نظر گرفته می‌شود.
- این مرحله یکی از نقاط کلیدی جلوگیری از خطای تشخیص است؛ چون در سکوت، خودهمبستگی می‌تواند قله‌های تصادفی تولید کند.

۳.۵ محاسبه خودهمبستگی و استخراج lag

محاسبه خودهمبستگی برای تمام lag ها هزینه‌بر است. ما فقط روی lag های LAG_MIN . . LAG_MAX جمع را انجام می‌دهیم:

$$R_x(l) = \sum_{n=l}^{N-1} x[n] x[n-l]$$

برای جلوگیری از سرریز:

- $x[n]$ در int16_t نگه‌داری شده،
- اما جمع $R_x(l)$ در int64_t انجام می‌شود.

۴.۵ روش‌های مختلف آستانه‌گذاری (Thresholding) که امتحان شد

یکی از مهم‌ترین چالش‌های پروژه، انتخاب آستانه‌ای بود که:

- در سکوت یا نویز کم، تشخیص غلط ندهد،
- در صدای ضعیف، گام را از دست ندهد،
- نسبت به تغییر دامنه (مثلاً اختلاف فایل‌های WAV) مقاوم باشد.

روش‌هایی که بررسی/پیاده‌سازی شد:

۱. آستانه درصدی از انرژی: تعریف threshold متناسب با انرژی فریم (یا حداکثر خودهمبستگی).
۲. Prominence Peak: معتبر دانستن قله‌ای که نسبت به همسایگی خود برجستگی کافی دارد.
۳. آستانه ثابت دستی: چندین مقدار ثابت در تست‌های مختلف امتحان شد.
۴. روش YIN (ایده جایگزین): استفاده از تابع اختلاف و CMNDF و آستانه روی CMNDF.

در نسخه نهایی (مطابق کد ارائه‌شده)، پس از عبور از تست سکوت، به جای «اولین قله غیر صفر»، lag متناظر با بیشینه قدر مطلق خودهمبستگی در بازه انسانی انتخاب شده است. این کار در عمل در شبیه‌سازی Proteus پایدارتر از آستانه‌گذاری حساس روی قله اول بود.

۵.۵ نگاشت فرکانس به نت موسیقی

برای نمایش نت، یک جدول از فرکانس نت‌ها (بر حسب centi-Hz) تعریف شده و نزدیک‌ترین مقدار انتخاب می‌شود:

$$\text{note} = \arg \min_i |f_i - f_0|$$

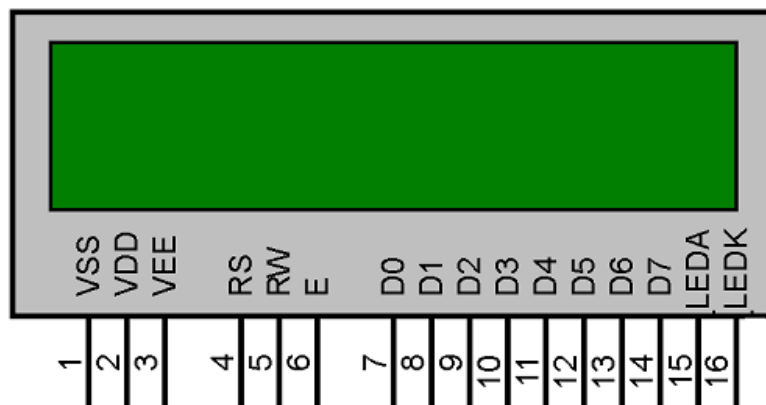
این روش ساده است و برای نیاز پروژه کافی بود؛ (در نسخه‌های حرفه‌ای‌تر می‌توان نیم‌پرده‌ها، اکتاو و خطای سنت را نیز محاسبه کرد.)

۶ LCD کاراکتری: پیاده‌سازی بدون کتابخانه و چالش زمان‌بندی (مهم‌ترین بخش عملی)

۱.۶ مروری بر $LCD\ 16x2$ و پایه‌ها

LCD های متنی رایج (کنترلر $HD44780$ یا سازگار) معمولاً ۱۶ پایه دارند. پایه‌های مهم:

- **RS**: انتخاب دیتا/دستور،
- **R/W**: خواندن/نوشتن (در پروژه غالباً روی GND برای نوشتن ثابت می‌شود)،
- **E**: Enable (پالس حیاتی برای ثبت داده)،
- **V₊..D₊D₊..D₊**: باس داده (در مد ۴ بیتی فقط $D4..D7$ استفاده می‌شود).



شکل ۳: پین‌آوت $LCD\ 16x2$

۲.۶ دو نسخه LCD : مد ۸ بیتی و مد ۴ بیتی

در پروژه دو نسخه درایور LCD بررسی شد:

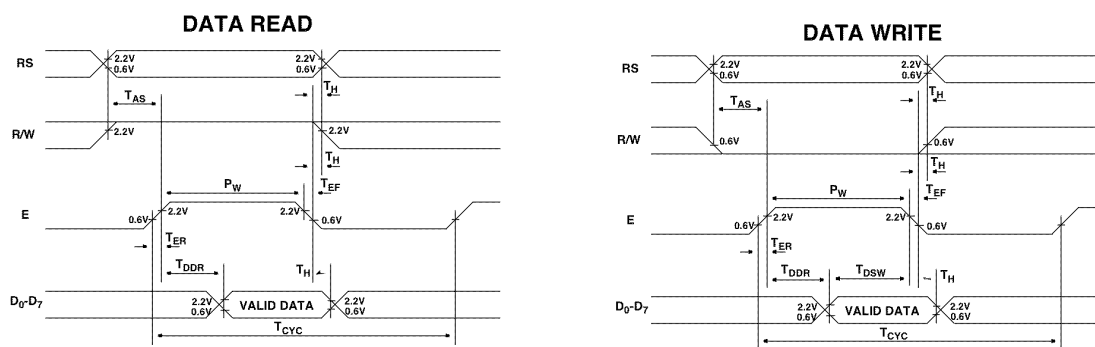
مقایسه دو مد LCD و علت اصلی خطاها

- مد ۸ بیتی: سریع‌تر و ساده‌تر از نظر توالی ارسال (یک بایت کامل)، اما مصرف پین زیاد.
- مد ۴ بیتی: نصف پین‌ها (فقط D4..D7) اما حساسیت به زمان‌بندی بیشتر است چون هر بایت در دو نیم‌بایت ارسال می‌شود.
- مشکل اصلی هر دو نسخه در عمل: زمان‌بندی (Timing). اگر پالس Enable، زمان‌های Setup/Hold دیتا، و Delay بین دستورات رعایت نشود:
- کاراکترها خراب/جابجا نمایش داده می‌شوند،
- LCD قفل می‌کند،
- یا دستوراتی مثل Clear به درستی اعمال نمی‌شوند.

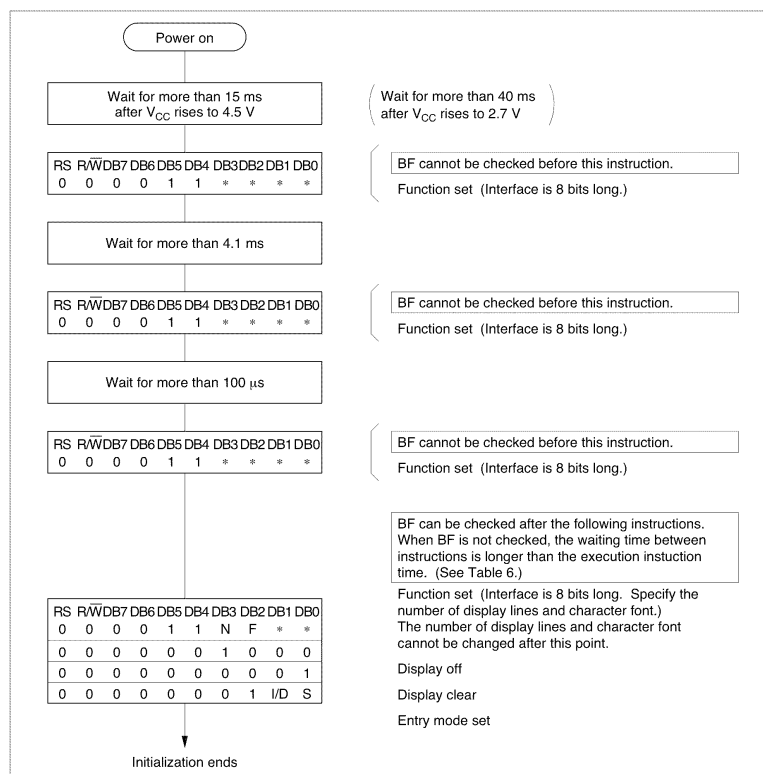
۳.۶ چرا زمان‌بندی در Proteus سخت‌تر شد؟

- در Proteus چند عامل باعث شد حساسیت زمانی بیشتر دیده شود:
- مدل LCD ممکن است Flag Busy را دقیق شبیه‌سازی نکند.
 - HAL_Delay میلی‌ثانیه‌ای است و برای پالس‌های چند میکروثانیه کافی نیست.
 - سرعت شبیه‌سازی/CPU و Scheduling داخلی Proteus می‌تواند باعث جیت‌ر زمانی شود.
- پس راه‌حل پایدار این بود که:
- از تاخیر میکروثانیه‌ای دقیق استفاده کنیم،
 - پالس Enable را با حداقل $1-2\mu s$ نگه داریم،
 - بعد از هر ارسال نیم‌بایت، حداقل $40-50\mu s$ مکث داشته باشیم،
 - بعد از دستوره‌ای سنگین مثل Clear/Home حداقل $2ms$ صبر می‌نماییم.

۴.۶ مرجع زمان‌بندی و توالی راه‌اندازی



شکل ۴: نمودار زمان‌بندی خواندن/نوشتن LCD (Setup/Hold و پالس Enable)



شکل ۵: توالی راه‌اندازی اولیه LCD (Power-On Initialization)

۵.۶ راه‌حل نهایی زمان‌بندی: استفاده از DWT برای delay میکروثانیه

در کد پروژه، برای رسیدن به تاخیر میکروثانیه‌ای:

- شمارنده سیکل **DWT->CYCCNT** فعال شد،
- تابع `lcd_delay_us()` نوشته شد،
- پالس Enable با `lcd_pulse_e()` دقیق تولید شد.

این دقیقاً همان بهینه‌سازی‌ای بود که مشکل عدم ثبات نمایشگر را در شبیه‌سازی تا حد قابل قبولی برطرف کرد/

خلاصه سریع چالش‌ها

۱. چالش زمان‌بندی LCD (بحرانی‌ترین): حل با *delay* میکروثانیه‌ای دقیق + رعایت توالی *init*.
۲. **Trigger** تایمیری *ADC* و تنظیمات صحیح: خاموش بودن *ContinuousConvMode* و تنظیم درست *TRGO/EOC*.
۳. آستانه‌گذاری (*Threshold*) برای تشخیص قله معتبر: تست چند روش (*energy*، درصدی، *prominence*، دستی، *YIN*).
۴. عدم دسترسی به سخت‌افزار و وابستگی به *Proteus*: نیاز به طراحی تست‌های دقیق *WAV* و دیباگ *UART*.
۵. جلوگیری از سرریز و خطای عددی: استفاده از *int64* برای خودهمبستگی و حذف *DC*.

۱.۷ چالش ۱: زمان‌بندی LCD (مد ۴ بیتی) — چرا اینقدر مهم بود؟

در مد ۴ بیتی، هر بایت در دو مرحله ارسال می‌شود:

Byte → (*HighNibble*) → (*LowNibble*)

اگر بین این دو ارسال یا در پالس *Enable* کوچک‌ترین خطا باشد:

- نیم‌بایت‌ها جابجا تفسیر می‌شوند،
- دستورها ناقص می‌رسند،
- *LCD* وارد حالت نامعلوم می‌شود.

چیزی که در عمل دیدیم:

- در برخی تست‌ها *LCD* فقط یک بار درست بالا می‌آمد و بعد قفل می‌کرد.
- گاهی فقط کاراکترهای آشغال (*garbage*) نمایش می‌داد.
- دستور *Clear* گاهی اجرا نمی‌شد یا باعث هنگ ظاهری می‌شد.

راه‌حل نهایی:

- ایجاد *Delay* میکروثانیه‌ای با *DWT*،
- بزرگ‌تر گرفتن *Delay* بعد از پالس *Enable* (مثلاً $50\mu s$)،
- اعمال *Delay* ویژه برای *0x01* و *0x02* (حداقل ۲، *ms*)،
- پایبندی به توالی راه‌اندازی (سه بار *0x03* سپس *0x02* و ...).

۲.۷ چالش ۲: ADC با Trigger تایمر — «چرا Continuous Conversion باید خاموش باشد؟»

وقتی ADC با رخداد تایمر Trigger می‌شود، اگر Continuous روشن باشد:

- ADC می‌تواند بدون توجه به Trigger پشت سرهم تبدیل انجام دهد،
- نرخ نمونه برداری واقعی از کنترل خارج می‌شود،
- یا در برخی تنظیمات، Trigger به شکل مورد انتظار عمل نمی‌کند.

پس در پروژه Continuous خاموش شد و ExternalTrigConv روی T2_TRGO قرار گرفت.

۳.۷ چالش ۳: Threshold — سخت‌ترین بخش DSP پروژه

این چالش دقیقاً همان جایی بود که خیلی راحت سیستم به «همیشه lag صفر» یا «تشخیص‌های ناپایدار» می‌رسید. چرا؟

- دامنه ورودی در فایل‌های WAV مختلف فرق می‌کند،
- در Proteus سطح نویز و کوانتیزاسیون می‌تواند متفاوت باشد،
- در سکوت، قله‌های تصادفی در خودهمبستگی دیده می‌شود،
- اگر threshold خیلی بالا باشد: همیشه خروجی صفر (No Signal)
- اگر خیلی پایین باشد: تشخیص غلط روی نویز

اقداماتی که انجام شد:

۱. دو معیار سکوت گذاشته شد: maxabs و energy.

۲. چند مدل threshold تست شد:

- درصدی از energy،
- peak prominence،
- threshold دستی،
- الگوریتم YIN (آستانه CMNDF).

۳. در نسخه پایدارتر، پس از عبور از سکوت، به جای حساسیت روی قله اول، بیشینه قدر مطلق خودهمبستگی در بازه انسانی انتخاب شد.

۴.۷ چالش ۴: نبود سخت‌افزار و تست در Proteus

چون سخت‌افزار واقعی در دسترس نبود:

- ورودی با فایل‌های WAV و منابع صوتی تست شد،
- UART به عنوان ابزار اصلی مشاهده وضعیت (note, f0, lag) استفاده شد،
- با تغییر سیستماتیک پارامترها (thresholds, Fs, N) پایداری بررسی شد.

۵.۷ چالش ۵: ساخت و آماده‌سازی فایل‌های WAV برای تست

برای تست دقیق، لازم بود WAV ها کنترل شده باشند:

- تک‌فرکانس (مثلاً $Hz 100$ ، $Hz 200$ ، $Hz 300$)،

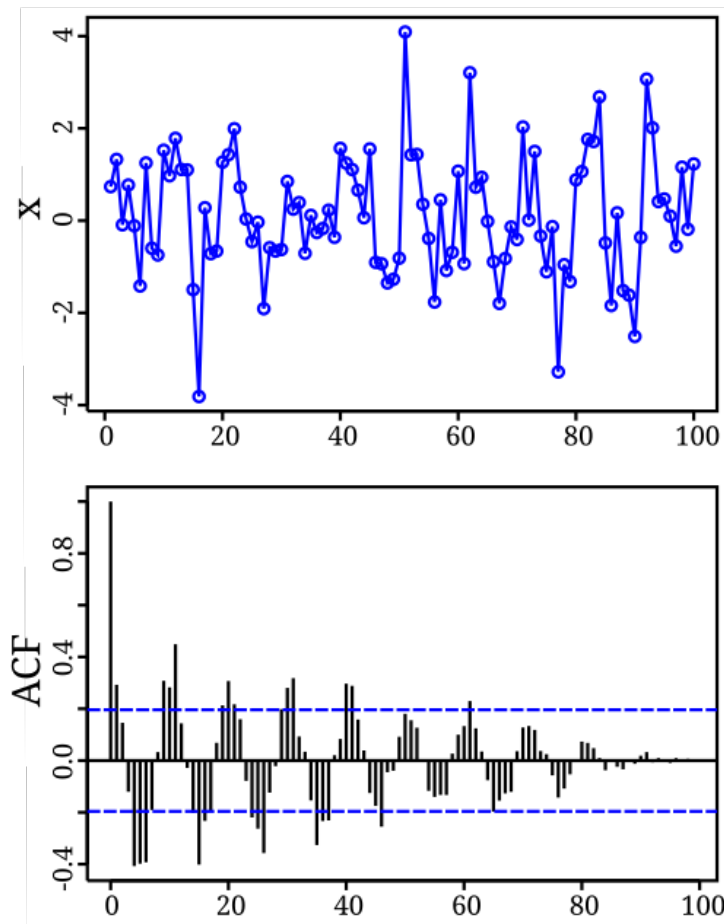
- Sweep یا چند نت پشت سر هم،

- فایل‌های ضبط شده صدای انسان (آواهای صدادار).

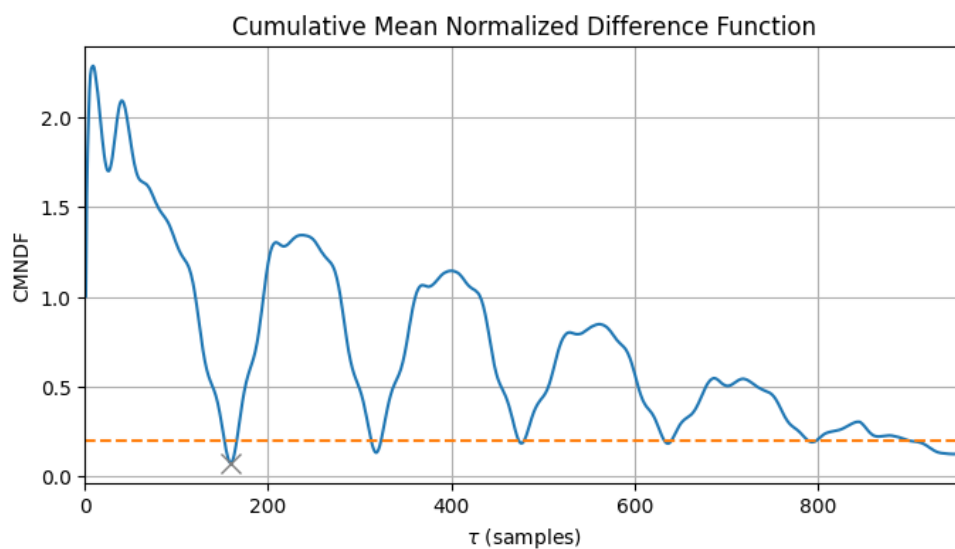
نکته مهم این بود که سطح دامنه و آفست ورودی باید با محدوده ADC سازگار باشد (تا اشباع رخ ندهد و دقت از دست نرود).

۸ تصاویر مفهومی تکمیلی

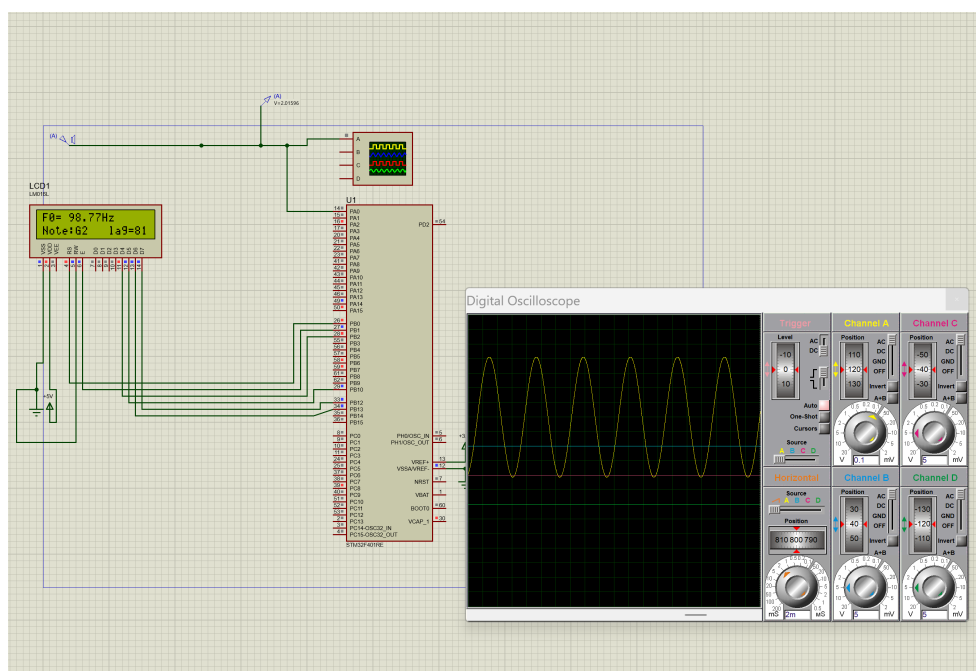
در این بخش چند شکل مفهومی مرتبط با الگوریتم‌ها آورده شده است.



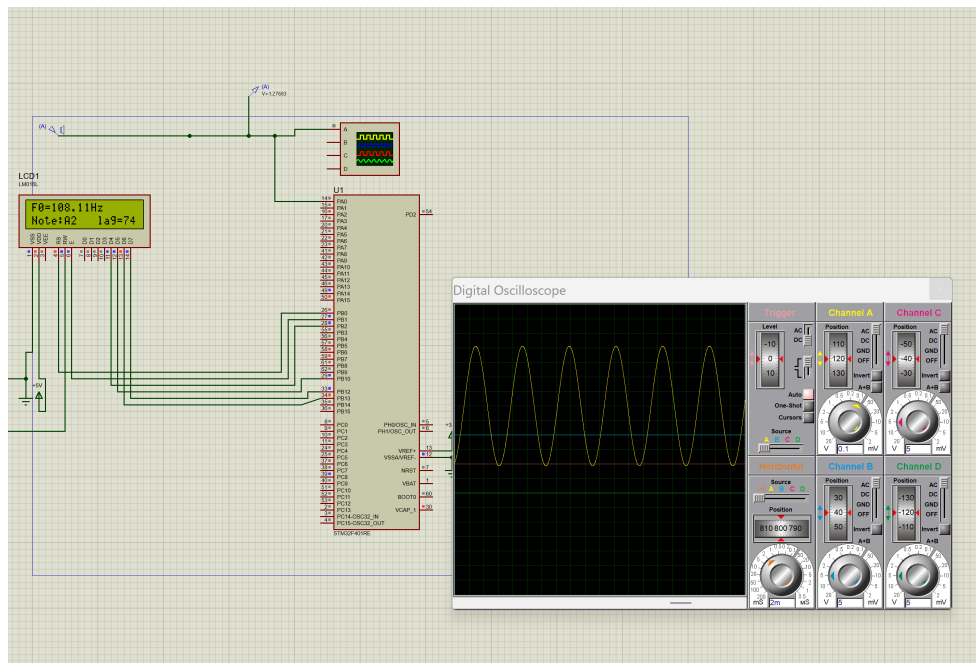
شکل ۶: نمونه شکل خودهمبستگی (ACF) و مشاهده قله‌ها/دوره



شکل ۷: نمودار $CMNDF$ در روش YIN و انتخاب τ



شکل ۸: خروجی حاصل از شبیه سازی ۱



شکل ۹: خروجی حاصل از شبیه سازی ۲

۹ نتایج، ارزیابی و مشاهده خروجی

در این پروژه خروجی به دو صورت ارزیابی شد:

- نمایش روی LCD: نمایش F_0 و نت متناظر (یا پیام *No Signal*).
- لاگ UART: چاپ فرکانس، نت و lag برای تحلیل و دیباگ.

۱.۹ سناریوهای تست پیشنهادی (که با Proteus هم قابل انجام است)

۱. سکوت: باید خروجی *No Signal* و — *Note*: باشد.
۲. سینوسی 100Hz : lag حدود ۸۰، نت نزدیک (با توجه به جدول نت‌ها).
۳. سینوسی 200Hz : lag حدود ۴۰.
۴. صدای انسان (آواهای ادار): خروجی باید در رنج 350 Hz — 85 Hz قرار بگیرد.

۱۰ جمع‌بندی

در این پروژه یک سیستم تشخیص گام بر پایه خودهمبستگی پیاده‌سازی شد که:

- نمونه‌برداری را با Trigger تایمر کنترل می‌کند،
- با پیش‌پردازش و تشخیص سکوت از خطاهای رایج جلوگیری می‌کند،
- با محاسبه خودهمبستگی در بازه lag انسانی، F_0 را تخمین می‌زند،
- و خروجی را به نت موسیقی نگاشت کرده و روی LCD نمایش می‌دهد.

مهم‌ترین تجربه عملی پروژه، کنترل دقیق زمان‌بندی LCD و تنظیم آستانه‌ها در شرایط متغیر دامنه/نویز (خصوصاً در Proteus) بود.

مراجع

- [1] Hamilton College (Phy245) — LCD Timing & Initialization (images used in report).
<https://physerver.hamilton.edu/courses/Spring17/Phy245/E%26CArticlesWeb/LCDDriver/>
- [2] STMicroelectronics — NUCLEO-F401RE board image. <https://www.st.com>
- [3] CircuitDigest — 16x2 LCD Pinout diagram. <https://circuitdigest.com>
- [4] ElectronicWings — LCD 16x2 interfacing (4-bit diagram). <https://www.electronicwings.com>
- [5] Wikimedia Commons — ACF example plot. https://commons.wikimedia.org/wiki/File:Acf_new.svg
- [6] ABSounds — YIN pitch estimation (CMNDF figure). <https://github.com/ABSounds/YIN-Pitch>
- [7] DeepBlueEmbedded — STM32 ADC external trigger diagrams. <https://deepblueembedded.com>